```
import numpy as np
from IPython.display import display, Math
import sympy as sp


x_0 = -2
x_k = 2
h_1 = 1.0
h_2 = 0.5
k = max(h_1, h_2) / min(h_1, h_2)
y = lambda x: 1 / (3 * x * x + 4 * x + 2)
x_var = sp.Symbol('x')
f = y(x_var)
eps = 5e-2
```

Метод прямоугольников

Определим функцию, которая считает приближённое значение интеграла по формуле метода прямоугольников для функции y на отрезке $[first_x, last_x]$ с фиксированным шагом h_step:

```
def get_rectangle(first_x, last_x, y, h_step):
    x_range = np.arange(first_x, last_x + h_step, h_step)

    return sum(
        h_step * y((x_range[i] + x_range[i + 1]) / 2)
        for i in range(len(x_range) - 1)
    )
```

Посчитаем приближённые значения интеграла по формуле метода прямоугольников для функции y на отрезке $[x_0, x_k]$ с шагами h_1 и h_2:

```
rectangle_h_1 = get_rectangle(x_0, x_k, y, h_1)
rectangle_h_1
```

```
np.float64(1.975292622928662)
```

```
rectangle_h_2 = get_rectangle(x_0, x_k, y, h_2)
rectangle_h_2
```

```
np.float64(1.8659258980586333)
```

Определим функцию, которая считает M - максимум по модулю от производной порядка $order$ для функции f на отрезке $[a, b]$:

```
def get_m(func, var, a, b, order):
    second_derivative = sp.diff(func, var, order)
    return max(
        abs(float(sp.maximum(second_derivative, var, sp.Interval(a, b)))),
        abs(float(sp.minimum(second_derivative, var, sp.Interval(a, b))))
    )
```

Найдём M_2 и M_4 для функции f на отрезке $[x_0, x_k]$:

```
m2 = get_m(f, x_var, x_0, x_k, 2)
m4 = get_m(f, x_var, x_0, x_k, 4)
```

Найдём верхнюю границу погрешности для метода прямоугольников для шагов h_1 и h_2:

```
rectangle_R_limit_h_1 = h_1 * h_1 * (x_k - x_0) * m2 / 24
rectangle_R_limit_h_2 = h_2 * h_2 * (x_k - x_0) * m2 / 24
```

Метод трапеций

Определим функцию, которая считает приближённое значение интеграла по формуле метода трапеций для функции y на отрезке $[first_x, last_x]$ с фиксированным шагом h_step:

```
def get_trapezoid(first_x, last_x, y, h_step):
    x_range = np.arange(first_x, last_x + h_step, h_step)

    return 0.5 * sum(
        h_step * (y(x_range[i]) + y(x_range[i + 1]))
        for i in range(len(x_range) - 1)
    )
```

Посчитаем приближённые значения интеграла по формуле метода трапеций для функции y на отрезке $[x_0, x_k]$ с шагами h_1 и h_2:

```
trapezoid_h_1 = get_trapezoid(x_0, x_k, y, h_1)
trapezoid_h_1
```

```
np.float64(1.7171717171717173)
```

```
trapezoid_h_2 = get_trapezoid(x_0, x_k, y, h_2)
trapezoid_h_2
```

```
np.float64(1.8462321700501896)
```

Найдём верхнюю границу погрешности для метода трапеций для шагов h_1 и h_2:

```
trapezoid_R_limit_h_1 = h_1 * h_1 * (x_k - x_0) * m2 / 12
trapezoid_R_limit_h_2 = h_2 * h_2 * (x_k - x_0) * m2 / 12
```

Метод Симпсона

Определим функцию, которая считает приближённое значение интеграла по формуле метода Симпсона для функции y на отрезке $[first_x, last_x]$ с фиксированным шагом h_step:

```
def get_simpson(first_x, last_x, y, h_step):
    x_range = np.arange(first_x, last_x + h_step, h_step)

    return (
        y(first_x) +
        y(last_x) +
        2 * sum(
            y(x_range[i])
            for i in range(1, len(x_range) - 2)
        ) +
        4 * sum(
            y((x_range[i - 1] + x_range[i]) * 0.5)
            for i in range(1, len(x_range) - 1)
        )
    ) * h_step / 6
```

Посчитаем приближённые значения интеграла по формуле метода Симпсона для функции y на отрезке $[x_0, x_k]$ с шагами h_1 и h_2:

```
simpson_h_1 = get_simpson(x_0, x_k, y, h_1)
simpson_h_1
```

```
np.float64(1.807017543859649)
```

```
simpson_h_2 = get_simpson(x_0, x_k, y, h_2)
simpson_h_2
```

```
np.float64(1.8297342810710984)
```

Найдём верхнюю границу погрешности для метода Симпсона для шагов h_1 и h_2:

```
simpson_R_limit_h_1 = h_1 * h_1 * h_1 * h_1 * (x_k - x_0) * m4 / 180
simpson_R_limit_h_2 = h_2 * h_2 * h_2 * h_2 * (x_k - x_0) * m4 / 180
```

Метод Рунге-Ромберга-Ричардсона

Определим функцию, которая считает приближённое значение погрешности по формуле метода Рунге-Ромберга-Ричардсона для приближённых значений f_h и f_{kh} с порядком точности p:

```
def get_runge(f_h, f_kh, p, k_coef):
    return (f_h - f_kh) / (k_coef ** p - 1)
```

Найдём приближённые значения погрешности по формуле метода Рунге-Ромберга-Ричардсона для приближённых значений, полученных по формулам методов прямоугольников, трапеций и Симпсона для шагов h_1 и h_2:

```
runge_rectangle = get_runge(rectangle_h_2, rectangle_h_1, 2, k)
runge_trapezoid = get_runge(trapezoid_h_2, trapezoid_h_1, 2, k)
runge_simpson = get_runge(simpson_h_2, simpson_h_1, 4, k)
```

```
display(runge_rectangle, runge_trapezoid, runge_simpson)
```

```
np.float64(-0.03645557495667625)
np.float64(0.043020150959490744)
np.float64(0.0015144491474299601)
```

Найдём приближённые значения интеграла, улучшенные по формуле метода Рунге-Ромберга-Ричардсона для приближённых значений, полученных по формулам методов прямоугольников, трапеций и Симпсона для шагов h_1 и h_2:

```
rectangle_integral = rectangle_h_2 + runge_rectangle
print("Метод прямоугольников:")
display(Math(rf"F = \int_a^b f(x)\,dx = {rectangle_integral}"))
```

```
Метод прямоугольников:
```

$$F = \int_a^b f(x)\,dx = 1.8294703231019571$$

```
trapezoid_integral = trapezoid_h_2 + runge_trapezoid
print("Метод трапеция:")
display(Math(rf"F = \int_a^b f(x)\,dx = {trapezoid_integral}"))
```

```
Метод трапеция:
```

$$F = \int_a^b f(x)\,dx = 1.8892523210096803$$

```
simpson_integral = simpson_h_2 + runge_simpson
print("Метод прямоугольников:")
display(Math(rf"F = \int_a^b f(x)\,dx = {simpson_integral}"))
```

```
Метод прямоугольников:
```

$$F = \int_a^b f(x)\,dx = 1.8312487302185283$$

Валидация

Проверим, что приближённые значения погрешности, полученные по формуле метода Рунге-Ромберга-Ричардсона для приближённых значений, полученных по формулам методов прямоугольников, трапеций и Симпсона для шагов h_1 и h_2, не меньше верхних границ погрешности для этих методов для шагов h_1 и h_2:

```
rectangle_R_limit_h_1 >= runge_rectangle and rectangle_R_limit_h_2 >= runge_rectangle
```

```
np.True_
```

```
trapezoid_R_limit_h_1 >= runge_trapezoid and trapezoid_R_limit_h_2 >= runge_trapezoid
```

```
np.True_
```

```
simpson_R_limit_h_1 >= runge_simpson and simpson_R_limit_h_2 >= runge_simpson
```

```
np.True_
```

Посчитаем точное значение интеграла функции f на отрезке $[x_0, x_k]$ и сравним его с приближёнными значениями, полученными по формулам методов прямоугольников, трапеций и Симпсона для шагов h_1 и h_2:

```
exact = float(sp.integrate(f, (x_var, x_0, x_k)))

display(Math(rf"F = \int_a^b f(x)\,dx = {exact}"))
```

$$F = \int_a^b f(x)\,dx = 1.8574186872187475$$

```
display(
    np.isclose(rectangle_integral, exact, eps),
    np.isclose(trapezoid_integral, exact, eps),
    np.isclose(simpson_integral, exact, eps)
)
```

```
np.True_
np.True_
np.True_
```

Посчитаем абсолютные погрешности для приближённых значений, полученных по формулам методов прямоугольников, трапеций и Симпсона для шагов h_1 и h_2:

```
print(
    f"Абсолютные погрешности:\n"
    f"Метод прямоугольников: {abs(rectangle_integral - exact)};\n"
    f"Метод трапеций: {abs(trapezoid_integral - exact)};\n"
    f"Метод Симпсона: {abs(simpson_integral - exact)};\n"
)
```

```
Абсолютные погрешности:
Метод прямоугольников: 0.02794836411679036;
Метод трапеций: 0.03183363379093285;
Метод Симпсона: 0.02616995700021918;
```